



Automatización de procesos con n8n

1º Administración de Sistemas Informáticos en Red

Víctor Palanca Pesudo

IES Juan de Garay

2026

Índice

1. Introducción
 - 1.1 Fundamentación teórica
 - 1.2 Contexto, motivación y objetivos generales
2. Metodología y herramientas empleadas
3. Talleres
 - 3.1 Taller 1: Pokemon Scraper
 - 3.1.1 Objetivo
 - 3.1.2 Requisitos previos
 - 3.1.3 Implementación
 - 3.1.4 Resultados
 - 3.1.5 Retos encontrados
 - 3.2 Taller 2: Wikipedia Agent
 - 3.2.1 Objetivo
 - 3.2.2 Requisitos previos
 - 3.2.3 Implementación
 - 3.2.4 Resultados
 - 3.2.5 Retos encontrados
 - 3.3 Taller 3: Sistema RAG Estándar
 - 3.3.1 Objetivo
 - 3.3.2 Requisitos previos
 - 3.3.3 Implementación
 - 3.3.4 Resultados
 - 3.3.5 Retos encontrados
 - 3.4 Taller 4: Telegram BOT
 - 3.4.1 Objetivo
 - 3.4.2 Requisitos previos
 - 3.4.3 Implementación
 - 3.4.4 Resultados
 - 3.4.5 Retos encontrados
4. Integración de los talleres
5. Conclusión
6. Bibliografía

1. Introducción

1.1 Fundamentación teórica

En nuestra sociedad, las personas realizamos constantemente tareas y procesos repetitivos. Concretamente, adentrándonos en el terreno empresarial, la eficiencia y la productividad muchas veces disminuyen debido a esto, ya que la repetición supone una pérdida enorme de tiempo, dinero y esfuerzo. Un ejemplo, son las pequeñas y medianas empresas en España, que pasan la mayor parte del tiempo buscando información, realizando informes, enviando correos electrónicos, copiando información a una hoja de cálculo y enviando avisos. Y ya no solo estas, incluso las grandes empresas sufren de esto. Necesitan automatizar tareas logísticas, financieras, y en empresas tecnológicas, gestionar la gran cantidad de datos y sistemas que manejan, además de reducir errores. Es ahí, donde las herramientas informáticas de automatización de tareas pueden ser realmente útiles.

Existen diversas herramientas para automatizar procesos, entre las que están Zapier, Make, UiPath, Microsoft Power Automate y n8n. Sin embargo, es en esta última en la que nos vamos a centrar ya que tiene multitud de ventajas.

La herramienta de automatización n8n es un modelo No-code de código abierto con licencia propietaria que permite conectar más de 400 diferentes aplicaciones (*Sass*), bases de datos y APIs mediante un sistema visual de nodos. A diferencia de los competidores comentados anteriormente, n8n no es solo una plataforma en la nube, sino un software que se puede instalar en un servidor propio. Esto es una gran ventaja debido a que los datos tienen un extra de seguridad al no tener que pasar por servidores externos y además es más económico porque solo se incurre en gastos relacionados con la infraestructura del servidor.

Además, una de las grandes características es que tiene agentes de IA integrados y personalizados, lo que permite crear chatbots, generar resúmenes o extraer información automáticamente. Por otra parte, tiene alta escalabilidad, puesto que procesa más de 200 tareas por segundo en una sola instancia.

En cuanto a su aplicación, esta se puede emplear en multitud de escenarios, desde gestión de contactos, sincronización de bases de datos, seguimiento de tareas, automatización de scripts, monitorización de procesos, y más.

Sin embargo, también tiene algunas desventajas, y es su curva pronunciada de aprendizaje, su número inferior de integraciones pre-configuradas que sus competidores, el mantenimiento

regular al alojarla en un servidor propio, y su mayor consumo de hardware debido a su alta velocidad de ejecución.

En cuanto a su funcionamiento, n8n se basa en la idea de que cada proceso puede segmentarse en pasos lógicos (nodos), conectados entre sí formando un flujo o *workflow*. Este flujo empieza con un *trigger* o disparador, como puede ser el recibir un correo nuevo, y a partir de entonces se ejecutan tareas de forma encadenada, como el análisis del mensaje, el almacenamiento de los datos y el envío de una notificación. Entre los nodos existentes, están los *triggers*, que acabamos de comentar, los nodos de acción que ejecutan una acción concreta, los nodos lógicos que controlan el flujo de la ejecución con condiciones y bucles, los nodos de datos (que permiten filtrar o analizar los datos) y los nodos personalizados, entre otros más..

Por otra parte, n8n mueve la información en formato JSON, un lenguaje universal que emplea objetos clave/valor y que usan las aplicaciones para hablar entre sí. Esto es muy importante ya que su ligereza hace que ocupe poco espacio de memoria y por tanto las automatizaciones sean más rápidas.

1.2 Contexto, motivación y objetivos generales

Todas estas funcionalidades y nuevas herramientas que están surgiendo han dado pie a que me anime a participar en un proyecto muy interesante basado en diferentes talleres de los cuales voy a aprender cómo realizar multitud de automatización de tareas, a entender la lógica de los nodos, manejar estructuras JSON y conectar diferentes aplicaciones API, entre otros. Esto da pie a adquirir nuevo conocimiento, desarrollando mi curiosidad y mis aptitudes a la hora de trabajar.

2. Metodología y herramientas empleadas

Como acabo de comentar, a lo largo de 2 meses se irán desarrollando 4 talleres, con dificultad incrementada y cada uno con una temática diferente. Además, se desarrollará esta memoria, que servirá de guía para demostrar el conocimiento que he adquirido y cómo he ido desarrollando cada tarea.

Entre las herramientas empleadas están la herramienta de automatización de n8n versión 1.x self-hosted o cloud, que esta última es la que voy a utilizar y la plataforma de Github, que servirá para alojar mi repositorio e ir adjuntando mis prácticas e ir consultando la información

sobre estas que se me ha facilitado. Además, será necesario Ollama una plataforma de código abierto diseñada para descargar y ejecutar Grandes Modelos de Lenguaje (LLMs), con los modelos llama 3.2 y llama 3.1 para embeddings. Y finalmente, una cuenta de Google Cloud para tener acceso a las APIs y herramientas de Google.

3. Talleres

3.1 Taller 1 - Pokemon Scraper

3.1.1 Objetivo

El objetivo de esta práctica es automatizar la consulta y el almacenamiento de datos de Pokémons. Aprenderemos cómo el sistema, a través de sus nodos, es capaz de leer una tabla de google sheet con los ID de los pokémons y, a través del flujo, se filtran los que aún no tiene datos, se consulta una API, se extrae la información relevante, se vuelve a escribir en el documento y envía una notificación al correo electrónico.

3.1.2 Requisitos previos

Antes que nada, ha sido necesario crearme una cuenta de Google Cloud para acceder tanto a la API de Google Sheets como a la de Gmail. Una vez creada la cuenta, configuré mi cuenta para enlazar cualquier servicio, puse la app en público, creé un nuevo proyecto llamado n8n-sheets y n8n-mail y saqué el OAuth client ID. Esta, una credencial única asignada a cada aplicación por un proveedor de servicios, y que permite que aplicaciones externas como n8n se identifiquen y soliciten permisos para acceder a servicios de terceros en mi nombre. Posteriormente, configuré las APIs en n8n.

Por otra parte, necesitaremos acceso a la API de Pokemon (PokéAPI), la cuál no requiere de registro ni de key, ya que es pública y gratuita. Esta tiene como límite 100 peticiones por minuto, suficiente para workflows pequeños.

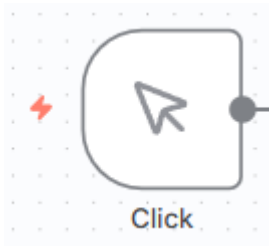
3.1.3 Implementación

En primer lugar, creé la hoja en blanco de n8n y luego el documento de google sheets con esta estructura, de manera que sólo estuviesen indicados los IDs de los Pokémon.

ID	Nombre	Tipo	Sprite Frontal	Sprite Posterior
1				
2				
4				
7				
15				
22				
32				
43				
53				
84				

- Nodo 1: Manual Trigger “Click”

En segundo lugar, configuré el primer nodo, el *trigger*, que en este caso va a ser un click manual. Este es perfecto para pruebas y se activa al pulsar *Execute workflow*.



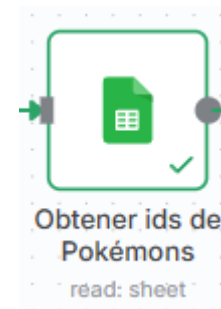
- Nodo 2: Google Sheets (Recuperar las filas del documento)

En segundo lugar, creé el nodo de Google Sheets con el objetivo de recuperar las filas del documento anteriormente creado. Para ello, vinculé mi documento y hoja al nodo y la configuré como “Get Rows” u obtener filas. En este caso no había ningún filtro adicional, por lo que no se tocó ninguna otra configuración. Como vemos, recupera el JSON con todas las filas y nombre de las columnas, y también podemos verlo en modo tabla.

Cabe destacar, que cada paso que voy haciendo en cada nodo voy haciéndole “pin” para guardar la información para el siguiente.

 The image shows a workflow editor interface. On the left, the configuration for a Google Sheets node titled "Obtener ids d..." is shown. It includes fields for "Credential to connect with" (Google Sheets account 6), "Resource" (Sheet Within Document), "Operation" (Get Row(s)), "Document" (From list, taller1), and "Sheet" (From list, Hoja 1). Below these are sections for "Filters" (Currently no items exist) and "Options" (No properties). On the right, the "OUTPUT" tab is active, displaying a JSON array of 10 items. The first three items are visible:

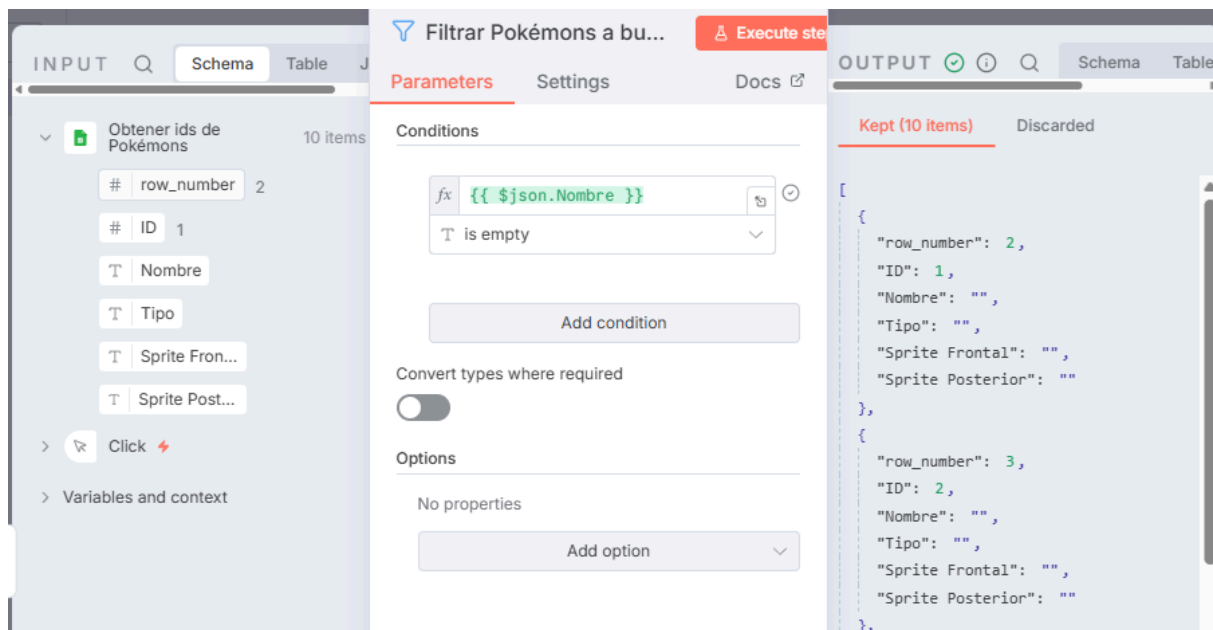

```
[
  {
    "row_number": 2,
    "ID": 1,
    "Nombre": "",
    "Tipo": "",
    "Sprite Frontal": "",
    "Sprite Posterior": ""
  },
  {
    "row_number": 3,
    "ID": 2,
    "Nombre": "",
    "Tipo": "",
    "Sprite Frontal": "",
    "Sprite Posterior": ""
  },
  {
    "row_number": 4,
    "ID": 4,
    "Nombre": "",
    "Tipo": "",
    "Sprite Frontal": "",
    "Sprite Posterior": ""
  },
  {
    "row_number": 5,
    "ID": 7,
    "Nombre": "",
    "Tipo": "",
    "Sprite Frontal": "",
    "Sprite Posterior": ""
  }
]
```



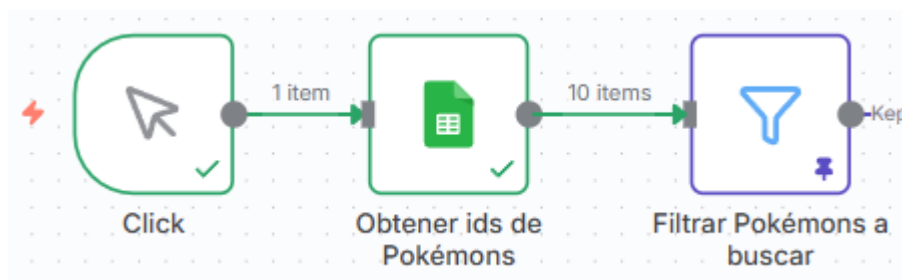
- **Nodo 3 - Filtrado de datos**

Este nodo, permite recuperar únicamente las filas en las que el Pokémon no tiene aún el nombre en el documento de Google Sheets, lo que indica que aún no ha sido procesado por la PokéAPI ni se ha recuperado su información. Esto ahorra espacio de memoria al no tener que volver a pedir todos los datos de nuevo.

En cuanto a su configuración, ponemos como condición que el nombre del Pokémon esté vacío (“is empty”). Basta con arrastrar el elemento para que se ajuste y como vemos, al ejecutar el paso, selecciona solo los que hemos indicado.

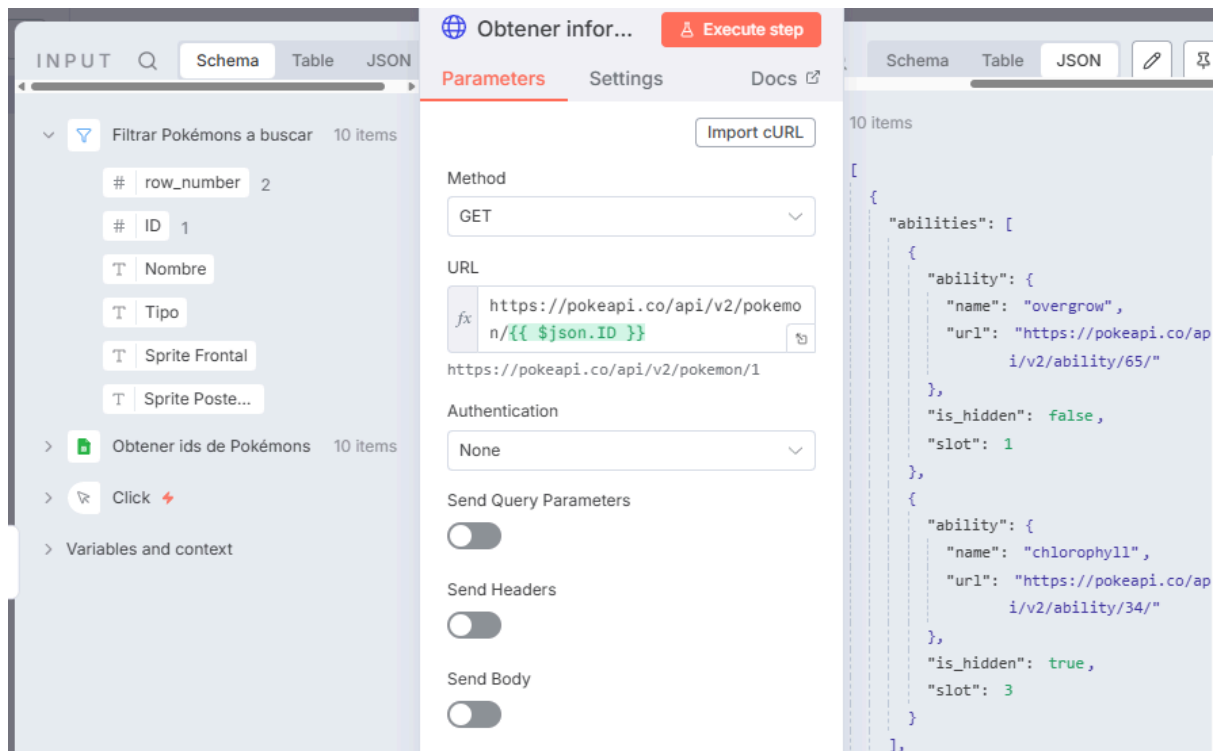


Como podemos ver en el esquema, en cada línea entre nodo y nodo van saliendo los ítems que se van recuperando del nodo anterior.



- Nodo 4 - HTTP Request (Extraer Pokémons a buscar)

A continuación, con este nodo vamos a solicitar recuperar la información del servidor de la PokéAPI mediante el método GET de HTTPS. Para ello, conociendo la url de la API, la insertamos en el sitio indicado y arrastramos el elemento por el que busca la aplicación (puede ser por el nombre del Pokémon o por el ID pero nosotros vamos a optar por el segundo). Ponemos la autenticación en “none” ya que al ser una API pública no necesitamos ninguna “key”. No cambiamos ningún parámetro más.



Como vemos, el servidor nos ha devuelto todos los datos de los Pokémons que hemos solicitado, pero eso no es lo que queremos, ya que queremos unos datos en concreto, por lo que pasamos al siguiente nodo.

- Nodo 5 - Edit Fields (Extraer datos del Pokémon)

Este nodo sirve para filtrar de la información que hemos recuperado, los 5 campos que queremos para que estén listos para escribirse en el Google Sheets. Para ello, ponemos el modo en “Manual Mapping” para configurar los campos uno a uno usando la interfaz visual. Luego, ponemos un nombre identificativo al dato recibido, ponemos el tipo de dato y arrastramos del esquema el campo que queremos.

The screenshot displays the configuration of the 'Extraer datos' (Extract Data) node in a workflow tool. The node is set to 'Manual Mapping' mode. The left panel shows the 'INPUT' schema with fields like 'abilities', 'name', 'url', 'is_hidden', and 'slot'. The middle panel shows the 'Fields to Set' configuration where four fields are mapped: 'identificador' (Number, mapped to '\$json.id'), 'nombre' (String, mapped to '\$json.name'), 'tipo' (String, mapped to '\$json.types[0].type.name'), 'imagen frontal' (String, mapped to '\$json.sprites.front_default'), and 'imagen trasera' (String, mapped to '\$json.sprites.back_default'). The right panel shows the 'OUTPUT' schema with the resulting JSON array containing data for Bulbasaur, Ivysaur, and Charmander.

- Nodo 6 - Google Sheets update (Actualizar información de los Pokémons)

Como en el segundo nodo, lo configuramos para que acceda al documento. Y en su configuración, ponemos que la operación sea actualizar las filas y “Match Each Column Manually” para decirle a la herramienta qué columna de la base de datos usar como identificador único para saber qué fila modificar. En este caso, los configuramos para que use el ID. Luego, ponemos los valores que queremos que actualice.

Actualizar inf... Execute step

Parameters Settings Docs

From list

Sheet: From list

Mapping Column Mode: Map Each Column Manually

Column to match on: ID

The column to use when matching rows in Google Sheets to the input items of this node. Usually an ID.

Values to Update

- ID (using to match): `{{ $json.identificador }}`
- Nombre: `{{ $json.nombre }}`
- Tipo: `{{ $json.tipo }}`
- Sprite Frontal: `{{ $json['imagen frontal'] }}`
- Sprite Posterior: `{{ $json['imagen trasera'] }}`

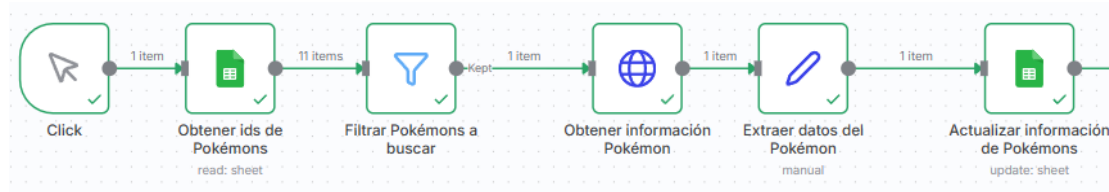
OUTPUT 10 items

```
[
  {
    "ID": 1,
    "Nombre": "bulbasaur",
    "Tipo": "grass",
    "Sprite Frontal": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/1.png",
    "Sprite Posterior": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/back/1.png"
  },
  {
    "ID": 2,
    "Nombre": "ivysaur",
    "Tipo": "grass",
    "Sprite Frontal": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/2.png",
    "Sprite Posterior": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/back/2.png"
  }
]
```

A continuación, al ejecutar el paso, podemos ver cómo se han escrito ya los elementos que queríamos en el Google Sheets.

ID	Nombre	Tipo	Sprite Frontal	Sprite Posterior		
1	bulbasaur	grass	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		
2	ivysaur	grass	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		
4	charmander	fire	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		
7	squirtle	water	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		
15	beedrill	bug	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		
22	fearow	normal	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		
32	nidoran-m	poison	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		
43	oddish	grass	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		
53	persian	normal	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		
84	doduo	normal	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s		

Si ponemos otro ID, podemos ver cómo funciona: pasan los 11 ítems de la base de datos pero solo se queda con el que carece de nombre y a partir de ahí sigue el flujo.



85	dodrio	normal	https://raw.githubusercontent.com/PokeAPI/	https://raw.githubusercontent.com/PokeAPI/s
----	--------	--------	---	---

- **Nodo 7 - Aggregate (Agrupar resultados)**

Como queremos que posteriormente nos envíe un email cuando la operación se haya finalizado con los elementos actualizados vamos a agregarlos en uno solo para que no se envíe un correo por cada ítem. Para ello configuramos la agrupación únicamente con los campos que queremos y como vemos quedan agrupados.

Agrupar resul...

Execute step

Parameters

Settings

Docs

Aggregate

Individual Fields

Fields To Aggregate

Input Field Name

ID

Enter the field name as text

Rename Field

Input Field Name

Nombre

Enter the field name as text

Rename Field

Add Field To Aggregate

Options

OUTPUT

Schema

1

1 item

ID

ID[0]

1

ID[1]

2

ID[2]

4

ID[3]

7

ID[4]

15

ID[5]

22

ID[6]

32

ID[7]

43

ID[8]

53

ID[9]

84

ID[10]

85

Nombre

T Nombre[0]

bulbasaur

T Nombre[1]

ivysaur

T Nombre[2]

charmander

- Nodo 8 - Gmail (Enviar un correo de resultado)

Finalmente, el flujo envía un correo de resultado indicando que se ha completado la ejecución satisfactoriamente. Para ello, configuramos el elemento como un mensaje, con la operación enviar, la dirección de correo a la que enviar la información, el asunto, el tipo de mensaje y el mensaje (junto con las etiquetas JSON).

INPUT Schema Table JSON

Enviar correo ... Execute step

Parameters Settings Docs

Credential to connect with
Gmail account 7

Resource
Message

Operation
Send

To
vppinf2@gmail.com

Subject
Actualización pokemon

Email Type
HTML

Message

```
<h1> El proceso terminó </h1>
Los pokemons con id {{ $json.ID }} y nombre {{ $json.Nombre }} han sido actualizados.
```

Options

Table Data:

ID	Nombre
ID[0]	1
ID[1]	2
ID[2]	4
ID[3]	7
ID[4]	15
ID[5]	22
ID[6]	32
ID[7]	43
ID[8]	53
ID[9]	84
ID[10]	85

Nombre

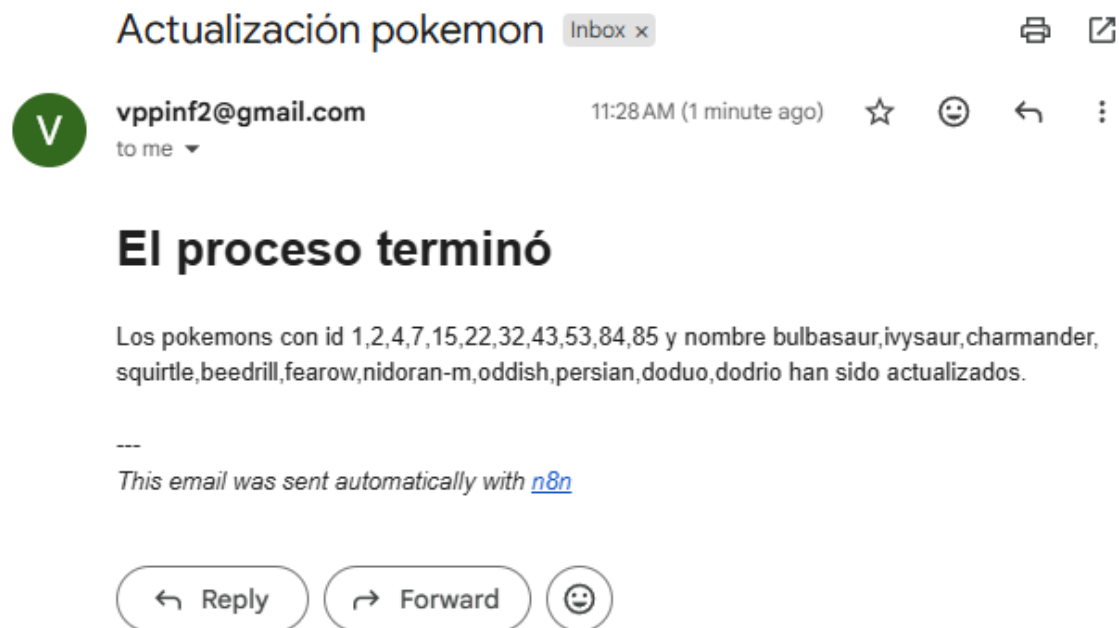
Nombre[0]	bulbasaur
Nombre[1]	ivysaur
Nombre[2]	charmander

3.1.4 Resultados finales

Finalmente, el flujo quedaría de la siguiente manera:



Y al ejecutarlo el correo se ha enviado con todos los datos correctamente:



3.1.5 Retos encontrados

Para comenzar, ha sido realmente fácil configurar este flujo. Sin embargo, lo más costoso ha sido conseguir las API keys de Google puesto que requerían suscripción y registro, y no funcionaba de forma correcta hasta que se iniciaba correctamente.

6. Bibliografía

<https://www.nocodehackers.es/herramientas-no-code/n8n>

<https://www.iebschool.com/hub/que-es-n8n-y-como-automatizar-tareas/>

<https://ikaue.com/blog-data/que-es-n8n-y-que-ventajas-tiene-sobre-otras-herramientas-de-automatizacion>

<https://codigoelectronica.com/blog/tipos-de-nodos-en-n8n>